

Four More Recursive Problems

Practice beyond Wednesday's examples

These four problems go with the recursion unit. Maze path finder is the one we work through together on Friday. The other three are here for extra practice, whenever you want more of them.

How to use this handout. For problems 1 and 2, the main method that builds the example maze or grid is given below, so your job is just the method underneath it. Every shell compiles and runs as shown, but returns a placeholder answer until you fill in the logic, including the base cases.

Before you write any code

Work out the base cases and the recursive idea for each problem as a group first. These three questions apply to all four problems below.

- What is the simplest input where you already know the answer without exploring any further?
- What does the problem look like one step smaller than where you are right now?
- How do the answers to those smaller versions combine into the answer you actually need?

1. Problem 1: maze path finder

There is a small grid made of open cells and walls, with a start cell marked S and an end cell marked E. Determine whether a path exists from S to E, moving only up, down, left, or right, without moving through a wall or stepping on a cell already visited during the current search.

```
S . . # .
# # . # .
. . . . .
. # # # .
. . . . E
```

This maze does have a path from S to E, so your finished program should print true.

Java

```
public static void main(String[] args) {
    char[][] maze = {
        {'S', '.', '.', '#', '.'},
        {'#', '#', '.', '#', '.'},
        {'.', '.', '.', '.', '.'},
        {'.', '#', '#', '#', '.'},
        {'.', '.', '.', '.', 'E'}
    };
    boolean[][] visited = new boolean[maze.length][maze[0].length];
```

```

        System.out.println(hasPath(maze, visited, 0, 0, 4, 4));
    }

    // TODO: implement this method, including the base cases we worked out as a class
    public static boolean hasPath(char[] [] maze, boolean[] [] visited,
                                   int row, int col, int endRow, int endCol) {
        return false;
    }

```

C++

```

const int COLS = 5;    // fixed for this exercise

int main() {
    char maze[5][COLS] = {
        {'S', '.', '.', '#', '.'},
        {'#', '#', '.', '#', '.'},
        {'.', '.', '.', '.', '.'},
        {'.', '#', '#', '#', '.'},
        {'.', '.', '.', '.', 'E'}
    };
    bool visited[5][COLS] = {};

    std::cout << std::boolalpha << hasPath(maze, visited, 5, 0, 0, 4, 4) << std::endl;
    return 0;
}

// TODO: implement this function, including the base cases we worked out as a class
bool hasPath(char maze[][COLS], bool visited[][COLS], int rows,
              int row, int col, int endRow, int endCol) {
    return false;
}

```

2. Problem 2: counting connected regions

Given a grid of land (L) and water (W) cells, and a starting cell that is land, count how many land cells can be reached from the start by moving up, down, left, or right through other land cells. This kind of counting is often called flood fill.

```

L L W W L
L W W L L
L W L L W
W W L L W
L L L W W

```

Starting at row 0, column 0, this region contains 4 cells, so your finished program should print 4.

Java

```

public static void main(String[] args) {
    char[][] grid = {
        {'L', 'L', 'W', 'W', 'L'},
        {'L', 'W', 'W', 'L', 'L'},
        {'L', 'W', 'L', 'L', 'W'},
        {'W', 'W', 'L', 'L', 'W'},
        {'L', 'L', 'L', 'W', 'W'}
    };
    boolean[][] visited = new boolean[grid.length][grid[0].length];

    System.out.println(regionSize(grid, visited, 0, 0));
}

// TODO: implement this method, including the base cases we worked out as a class
public static int regionSize(char[][] grid, boolean[][] visited, int row, int col) {
    return 0;
}

```

C++

```

const int COLS = 5;    // fixed for this exercise

int main() {
    char grid[5][COLS] = {
        {'L', 'L', 'W', 'W', 'L'},
        {'L', 'W', 'W', 'L', 'L'},
        {'L', 'W', 'L', 'L', 'W'},
        {'W', 'W', 'L', 'L', 'W'},
        {'L', 'L', 'L', 'W', 'W'}
    };
    bool visited[5][COLS] = {};

    std::cout << regionSize(grid, visited, 5, 0, 0) << std::endl;
    return 0;
}

// TODO: implement this function, including the base cases we worked out as a class
int regionSize(char grid[][COLS], bool visited[][COLS], int rows, int row, int col) {
    return 0;
}

```

3. Problem 3: Sierpinski triangle

Print a Sierpinski triangle pattern of asterisks, given an order n . A triangle of size 1 is a single filled cell. A triangle of size $2n$ is a smaller triangle of size n on top, and two more copies of that same smaller triangle side by side underneath it, with the mirror-image gaps that pattern leaves behind.

```

.*..
.**.

```

```
*.*.  
****
```

This is order 2, meaning a size of 4 (2 raised to the second power). A dot above stands in for a blank space, shown only so the pattern is easy to see on the page. Your finished program should reproduce this for order 2.

Java

```
// TODO: implement this method, including the base case, so that it  
// correctly identifies which cells belong to the triangle  
public static boolean isFilled(int row, int col, int size) {  
    return false;  
}  
  
// To print it: for a chosen order, let size = 1 << order, then for each  
// row and column from 0 to size - 1, print '*' when isFilled is true and  
// a space otherwise, with a newline after each row.
```

C++

```
// TODO: implement this function, including the base case, so that it  
// correctly identifies which cells belong to the triangle  
bool isFilled(int row, int col, int size) {  
    return false;  
}  
  
// To print it: for a chosen order, let size = 1 << order, then for each  
// row and column from 0 to size - 1, print '*' when isFilled is true and  
// a space otherwise, with a newline after each row.
```

4. Problem 4: coin change counting

Given a target amount and a list of coin denominations, count how many different ways there are to make that amount. The order the coins are picked in does not matter, and each denomination can be used as many times as needed.

With coins {1, 2, 5} and an amount of 5, there are 4 ways: 1+1+1+1+1, 1+1+1+2, 1+2+2, and 5. Your finished program should report 4 for this input.

Java

```
// TODO: implement this method, including the base cases we worked out as a class  
public static int countChange(int amount, int[] coins, int index) {  
    return 0;  
}
```

C++

```
// TODO: implement this function, including the base cases we worked out as a class
int countChange(int amount, int coins[], int numCoins, int index) {
    return 0;
}
```